BROWSER VÀ WEB SERVER GIAO TIẾP NHƯ THẾ NÀO?

MSc. Ngô Ngọc Đăng Khoa

BROWSER VÀ WEB SERVER GIAO TIẾP NHƯ THẾ NÀO?

NỘI DUNG BUỔI HỌC

01 Câu hỏi và mental model

02 Khái niệm và ví dụ

03 Bài tập

04 Nghiệm thu và trình bày

05 Tổng kết và đối chiếu

CÂU HỎI LỚN

> Khi người dùng nhập một URL, chuyện gì xảy ra giữa browser và web server?

MỤC TIÊU ĐẦU RA

Giải thích **BROWSER VÀ WEB SERVER** làm gì

01

Mô tả được một vòng **REQUEST → RESPONSE**

02

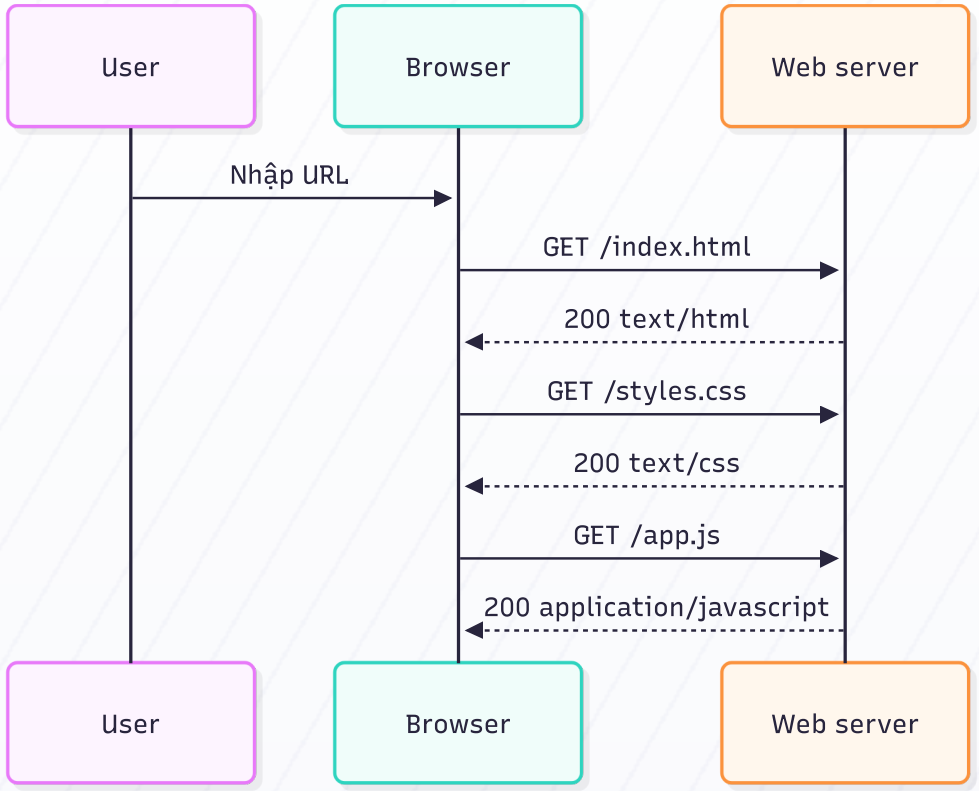
Phân biệt **STATIC, SSR VÀ CSR** ở mức mental model

03

Serve static site và dùng Network làm bằng chứng

04

MENTAL MODEL



WHAT — BROWSER VÀ WEB SERVER

Browser

- Biến URL thành HTTP request
- Nhận và diễn giải response
- Parse HTML/CSS, chạy JavaScript
- Hiển thị và nhận tương tác

Web server

- Lắng nghe request qua HTTP
- Ánh xạ URL tới resource
- Trả status, headers và body
- Ghi log để quan sát phía server

WHAT — MỘT VÒNG HTTP CƠ BẢN

REQUEST TỪ BROWSER	RESPONSE TỪ SERVER
Method + URL	Status code
Headers	Headers
Body nếu cần	Body: HTML, CSS, JS, ảnh...

```
GET /styles.css   →  200 OK
GET /missing.png →  404 Not Found
```

WHY — KHÔNG CHỈ MỞ FILE LOCAL?

- URL `http://...` buộc browser giao tiếp với một server
- Response có **status** và **Content-Type** để diễn giải tài nguyên
- URL path được tách khỏi đường dẫn vật lý trên một máy cụ thể
- Log và Network cho bằng chứng về ranh giới browser–server
- Cách chạy gần hơn với môi trường deploy thật

WHAT — MỘT PAGE LOAD TẠO NHIỀU REQUEST

```
GET /                  → index.html
      ├─ GET /styles.css
      ├─ GET /app.js
      └─ GET /images/avatar.webp
```

- HTML ban đầu **tham chiếu** các resource khác
- Mỗi resource có response và lỗi độc lập
- Trang "mất CSS" thường là request CSS sai, không phải HTML biến mất

WHAT / WHY / WHEN — STATIC SITE

- **WHAT:** HTML/CSS/JS đã tồn tại thành file trước request
- **WHY:** phục vụ đơn giản, nhanh và ít thành phần runtime
- **WHEN:** profile, landing page, tài liệu, nội dung xuất bản ít thay đổi
- Server đọc file phù hợp và trả bytes; không tạo HTML theo từng request

> "Static" nói về cách tạo/phục vụ tài nguyên, không có nghĩa là trang không có JavaScript.

WHAT / WHEN — STATIC, SSR VÀ CSR

MÔ HÌNH	HTML CHÍNH XUẤT HIỆN Ở ĐÂU?	THƯỜNG PHÙ HỢP KHI
Static	File có sẵn trước request	Nội dung publish ổn định
SSR	Server tạo cho request	HTML cần dữ liệu mới theo request
CSR	JavaScript dựng nhiều UI trong browser	Trải nghiệm app giàu tương tác

Tất cả vẫn dùng request/response; khác nhau ở **nơi và lúc tạo UI**.

WHY — RENDERING MODEL LÀ MỘT QUYẾT ĐỊNH

- Ảnh hưởng thời điểm người dùng thấy nội dung
- Quyết định server phải làm gì cho mỗi request
- Thay đổi lượng JavaScript và công việc trong browser
- Tác động cách debug: xem response HTML hay trạng thái runtime

> Không có mô hình “tốt nhất” nếu chưa biết requirement.

HOW — ĐỌC DEVTOOLS NETWORK

- **Name / URL:** browser đang xin tài nguyên nào?
- **Status:** server trả kết quả gì?
- **Type / Content-Type:** browser hiểu body là gì?
- **Initiator:** điều gì làm request xuất hiện?
- **Response:** body thực tế có đúng tài nguyên mong đợi?

HOW — NHỜ AGENT CẤU HÌNH SERVER

QUYẾT ĐỊNH — Server, thư mục site và địa chỉ truy cập

01

YÊU CẦU — Cấu hình tối thiểu và cách chạy/dừng

02

ĐỌC — Tìm mapping giữa URL path và document root

03

CHỨNG MINH — Dùng log, Network và một request lỗi

04

LỖI THƯỜNG GẶP

- Dùng development server thay cho Nginx/Caddy
- Nhầm **URL path** với đường dẫn file trên máy
- HTML trả 200 nhưng CSS/JavaScript lại 404
- Server trả sai Content-Type cho tài nguyên
- Chụp giao diện nhưng không lưu bằng chứng request/response

20–60 PHÚT

BÀI TẬP

HOST STATIC SITE CỦA BLOCK 1 BẰNG NGINX HOẶC CADDY.

ĐỀ BÀI

Mỗi nhóm phải:

- Đưa site từ Block 1 lên một web server local
- Truy cập site bằng URL qua HTTP
- Dùng Network để kể lại một page load
- Chứng minh server xử lý cả tài nguyên có và không tồn tại

NHÓM PHẢI TỰ QUYẾT ĐỊNH

Chọn **NGINX HOẶC CADDY**

01

Thư mục nào là document root

02

Dùng địa chỉ và cổng local nào

03

Cách tạo một request lỗi có chủ đích, không sửa source chính

04

Evidence nào đủ để giải thích luồng

05

YÊU CẦU TỐI THIỂU

- Không dùng development server của framework
- HTML, CSS và JavaScript được tải qua web server
- Trang hoạt động sau một lần reload sạch
- Network cho thấy request thành công và một request **404**
- Nhóm giải thích được URL nào ánh xạ tới resource nào

ĐẦU RA CỦA MỖI NHÓM

Source static site và file cấu hình server

01

Hướng dẫn ngắn để **RUN VÀ STOP**

02

Network evidence gồm document, asset và một request lỗi

03

Giải thích ngắn một cặp request/response đã quan sát

04

TRÌNH BÀY VÀ TỔNG KẾT

Hai nhóm ngẫu nhiên · 10 phút mỗi nhóm

KỊCH BẢN DEMO 10 PHÚT

Mở URL qua cổng web server

01

Reload và lọc request theo loại tài nguyên

02

Mở một response body trong Network

03

Gõ URL sai để chứng minh 404

04

ĐIỀU CẦN NHỚ

Browser bắt đầu bằng một **HTTP REQUEST**

01

Response mang status, headers và nội dung

02

Static server phục vụ file đã tồn tại

03

Network panel là bằng chứng cho luồng browser–server

04

BTVN TUẦN 1

Hoàn thiện sản phẩm

- Static profile site có responsive
- Plain CSS hoặc framework cơ bản
- Ít nhất một interaction JavaScript
- Host bằng Nginx hoặc Caddy

Nộp

- Source và server config
- Hướng dẫn run ngắn
- Minh chứng UI và Network
- Ghi nhận ngắn cách đã dùng agent

Q & A

Nộp artifact trước khi rời lớp